

Memoria

Se nos muestra un árbol de procesos donde cada proceso tiene una serie de page frames asignados. Explicar las siguientes situaciones:

- ¿Por qué todos los procesos de un sistema compartirían una página? (páginas del kernel o bibliotecas compartidas)

Es eficiente que todos los procesos compartan paginas de memoria.

Paginas de Kernel: Contiene codigo y estructuras esenciales para el funcionamiento del sistema operativo. Evitamos duplicar el mismo codigo en la memoria de cada proceso. El kernel mapea estas paginas como compartidas en el espacio de direcciones de cada proceso.

Bibliotecas compartidas: Las bibliotecas compartidas son fundamentales para evitar la duplicacion de codigo en memoria. Bibliotecas comunes como *libc* son usadas por casi todos los procesos del sistema.

En vez de cargar una copia de la biblioteca para cada proceso, se mantiene una copia en la memoria para que todos la utilicen.

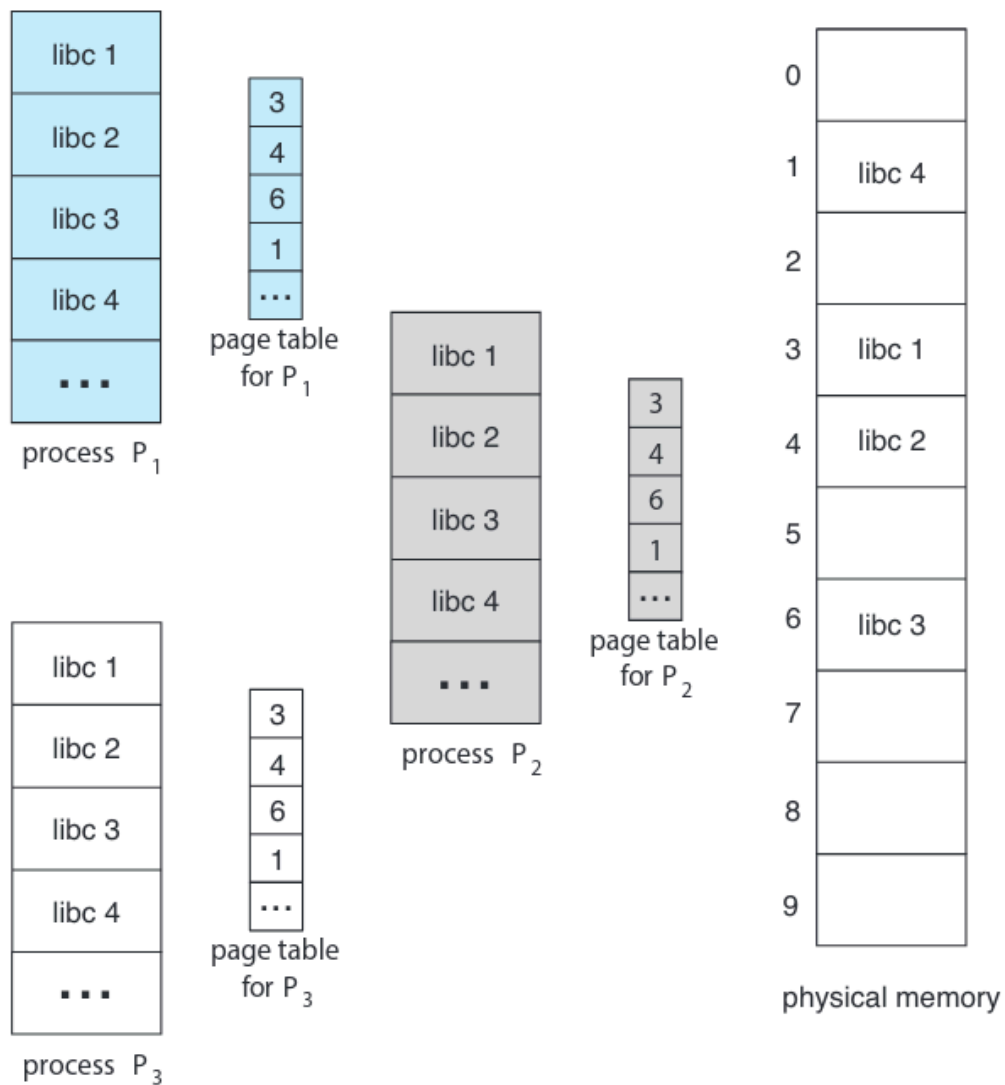


Figure 9.14 Sharing of standard C library in a paging environment.

- Hay 3 procesos donde se ve su memoria logica contigua y al lado su page table representando las direcciones fisicas.
- Las direcciones fisicas son direcciones contiguas en una tabla que al final terminan uniendo las fisicas como contiguas.

Generalmente, para ambos casos, se mapean como solo lectura. Si un proceso intenta modificarla se genera un page fault y se asigna una copia privada al proceso.

- ¿Por qué dos procesos específicos podrían compartir una página? (hablar de fork y copy-on-write)

Cuando hacemos un fork queremos un proceso que sea un clon del proceso padre, esto implica mismas variables, estado del stack, etc. Pero si son iguales no hace falta copiar las paginas para el proceso nuevo y tener datos repetidos en la memoria principal. Si alguno de los procesos modifica alguna de las paginas estas se duplican y queda un proceso con los datos originales y otro con los datos

modificados. (copy-on-write, copiamos recién cuando hubo una modificación y no arbitrariamente)

¿Para qué sirve la paginación de la memoria? ¿Qué ventajas tiene sobre utilizar direcciones físicas? Hablar sobre el tamaño de las páginas y cómo se relaciona con el tamaño de los bloques en disco. (hablar de fragmentación interna y fragmentación externa)

1. Sirve para resolver el problema de fragmentación externa. Uno de los problemas de la memoria es que dependiendo del algoritmo que usemos para asignar memoria (first-fit, best-fit o worst-fit) puede suceder que exista el espacio necesario para agregar un proceso nuevo pero que no sea contiguo impidiendo la carga de este. Con paginación simplemente le pedimos una página libre al kernel y que el proceso la interprete como contigua. De esta manera el límite será siempre la cantidad de páginas libres con respecto a un proceso nuevo (falta de espacio) y no necesariamente una mala decisión de algoritmo de asignación como antes. Ojo, esta solución genera fragmentación interna donde tal vez una página solo es usada por una fracción del proceso por lo que queda espacio de más inutilizado.
2. Podemos usar direcciones lógicas contiguas que en realidad no son contiguas en memoria física. De esta manera si el espacio total libre en memoria es mayor al proceso, tenemos asegurado que nuestro proceso va a poder ser asignado a memoria a diferencia de memoria física donde si hay 4gb libres pero son de a bloques contiguos pero separados de 1gb 1gb 2gb y mi proceso es de 3gb no podría asignarlo.
3. El tamaño de las páginas y el de los marcos de página en disco son iguales. De esta manera podemos tener una capacidad de *memoria swap* donde mandamos páginas de procesos al disco cuando la memoria principal está llena. Luego si la página es requerida por el proceso en ejecución, se la busca de disco, se la swapea con otra página en memoria y se la usa. Es una manera de estandarizar la memoria con el disco y tener una fácil interacción entre ambos.

La tabla de páginas es un mapa que traduce cada dirección lógica del proceso a su correspondiente dirección física, permitiendo la ilusión de contigüidad.

Un tamaño de página más grande reduce la cantidad de entradas en la tabla de páginas pero aumenta la fragmentación interna.

Un tamaño de página más pequeña minimizan la fragmentación interna pero generan más overhead en las tablas y más swaps entre memoria y disco.

¿Qué es un page fault y cómo se resuelve?

Un page fault sucede cuando un proceso intenta acceder a una página de memoria que no está actualmente cargada en la memoria principal.

Puede suceder porque la pagina fue movida a disco o todavia no fue cargada a la memoria principal o es una direccion invalida.

1. La mmu detecta que la pagina solicitada no esta en memoria. Interrupcion. (bit de presencia 0)
2. Se verifica si el acceso es valido. Si es valido significa que la pagina esta en el disco. Si no lo es, se genera un seg fault.
3. Se busca la pagina en el area de swap (disco) y se lo carga en un marco de pagina libre en la memoria ram.
4. Se actualiza la tabla de paginas del proceso.
5. Se reanuda el proceso.

¿Por qué puede pasar que tengamos muchos procesos en waiting, y cómo podría tratar de arreglarlo si no pudiese agregar memoria?

Podria pasar que los procesos esten esperando a que las paginas que necesitan para continuar su ejecucion sean buscadas del disco y swapeadas por paginas actuales porque no se encontraban en la memoria principal (*thrashing*), tengamos en cuenta que el disco es lento, muy leeeentooooo.

Soluciones:

- Reducir la cantidad de procesos activos, posponer su ejecucion. Se pueden suspender procesos menos prioritarios para liberar memoria.
- Cambiar el algoritmo de reemplazo de paginas para usar uno mas adecuado para el trabajo (FIFO, LRU, Clock).
- Cambiar politicas de scheduling como usar Shortest Job First para ayudar a disminuir los tiempos de espera en ciertos escenarios.
- Optimizar procesos para que usen menos memoria.
- Limitar la cantidad de proceso ejecutando simultaneamente.

El principal problema es el **thrashing** causado por la falta de memoria.